

LoadLead Platform E2E Audit v8

Date: 2026-08-23 **Baseline:** `main @ 3cd3eb9` **Prod backend:** `loadlead-backend-20260720172452` (Ready / Green) **Auditor:** Platform Engineering **Scope:** Full platform. Backend business logic, frontend, API + route paths, cross-environment parity (prod / staging / dev), live production smoke, authorization posture, money and compliance correctness.

1. Executive summary

The platform is in strong shape. The backend suite is fully green (898/898), production is healthy and hardened, the live authorization posture is fail-closed on every protected path probed, and the core marketplace race (first-accept-wins) is provably atomic. Every CRITICAL and HIGH from prior rounds (v1 through v7) that was spot-checked remains remediated in code and, where relevant, live in production.

This round went deeper into the money and capacity math and surfaced **one HIGH business-logic defect** that has not appeared in any prior audit: the accessorial detention-to-layover billing curve is **non-monotonic** under the default policy, so a load billed for detention can cost **dramatically less** by dwelling one minute longer. It is gameable and does not make sense as a billing model. A second, **MEDIUM** invariant break was found in the operational-capacity service, which computes weight in floating point rather than integer pounds.

Environment parity is largely intact. The notable gaps are a **staging table that was never applied** (`CapacityStateEvents`) and the **production frontend running behind** `main` by the landing-page fix that is already staged and awaiting promotion.

Verdict: SHIP-HEALTHY with two logic fixes to schedule. No active CRITICAL. F1 (accessorial non-monotonicity) should be fixed before accessorial billing is switched on broadly.

Severity tally

Sev	ID	Title	Area
HIGH	F1	Accessorial detention→layover billing is non-monotonic (charge drops as dwell rises)	Money / business logic
MEDIUM	F2	Operational-capacity math uses floating-point pounds (integer-pounds invariant break)	Capacity / invariant
MEDIUM	P1	Staging is missing the <code>CapacityStateEvents</code> DynamoDB table	Parity
MEDIUM	P2	Production frontend is behind <code>main</code> (landing mobile fix not promoted)	Parity / deploy
LOW	F3	AML gate holds a <code>pending</code> status even when <code>AML_REQUIRED</code> is off	Compliance edge
LOW	F4	Stale comments claim the reverse of the SEC-C1 fix (regression trap)	Doc hygiene / security
LOW	P3	Prod TF declares 2 Canopy COI tables not applied (deliberate, gated feature)	Parity (known)
INFO	P4	Staging backend is paused; staging EB carries out-of-band env drift	Parity / readiness

2. Method

- **Static + logic review** of the money, capacity, negotiation, verification/compliance, and authorization services, reading the actual code paths rather than trusting comments.
- **Full backend test suite** (`vitest run`) executed from `main` .
- **Live production smoke:** unauthenticated probes across the auth, IDOR-class, webhook, and public path surfaces, asserting fail-closed behavior.
- **Cross-environment parity:** `tofu plan` drift on prod and staging, the table-env parity checker, deployed-code-vs- `main` for both backend (EB version) and frontend (served bundle), and live health on prod and staging.

- **Contract check:** frontend API path inventory diffed against backend route mounts.
- **Frontend typecheck + build.**

All checks are code- and infrastructure-level. No production personal data was read or included in this report.

3. Baseline health (all GREEN)

Check	Result
Backend unit/integration suite	898 passed / 898 (114 files)
Frontend typecheck + build	Clean
Table-env parity checker	PASS (64 table vars; staging via <code>DYNAMODB_TABLE_PREFIX</code> , dev explicit)
Prod <code>/api/health</code>	<code>ok</code> , <code>productionHardened: true</code>
Prod backend deploy vs <code>main</code>	Current (<code>...20260720172452</code> = commit <code>0c8bf4b</code> , the latest backend-touching commit)
Money-write idempotency	Intact (<code>attribute_not_exists</code> on <code>advanceId</code> / <code>outcomeId</code>)
First-accept-wins lock	Atomic (<code>attribute_not_exists(loadId)</code> conditional put; loser gets 409)
Payout-intercept surfacing (SEC-6)	Intact (garnishment/levy/lien surfaced in payee routing)
Notification suppression seam	Wired (law-enforcement hold + restricted-entity push suppression)
Self-signup-ADMIN (SEC-C1)	Fixed in code (<code>resolvePlatformRole(null) → null</code> ; <code>requireStaffTier</code> denies)
Authorization never trusts JWT	Confirmed (<code>requireStaffTier</code> reads tier from a fresh DB get)
COI expiry	Handled (<code>expireDueCois</code> sweeper + expired flags in packet/decision)
FE ↔ BE path contract	Clean (every FE path first-segment maps to a BE mount)

Live production smoke (unauthenticated)

Path	Expected	Observed
POST /api/auth/login (bad creds)	401	401
POST /api/auth/signup (role=ADMIN)	403 beta wall	403
GET /api/maps/autocomplete	401	401
GET /api/driver/loadboard	401	401
GET /api/shipper/loads	401	401
GET /api/factoring/assignments	401	401
GET /api/admin/staff	401	401
GET /api/org/:id/members	401	401
GET /api/accessorials/charges	401	401
GET /api/capacity/state	401	401
POST /api/webhooks/canopy (unsigned)	401	401
POST /api/webhooks/didit (unsigned)	401	401
POST /api/support/inbound (bad body)	4xx not 500	400
GET /api/compliance/w9/render-check	200, rate-limited	200
GET /api/<unknown>	JSON 404	{"message":"Not found","statusCode":404}

Every protected path fails closed. No 500s on the malformed/webhook probes.

4. Findings

F1 — HIGH — Accessorial detention→layover billing is non-monotonic

Where: backend/src/services/accessorialCalc.ts:70 (computeAccessorialFromDwell).

What: When dwellMinutes > layoverThresholdMinutes , the engine bills **layover only** and discards the detention that accrued in the first threshold window (the code comment says "detention stops accruing once layover takes over"). Because layover is billed as started 24-hour periods at a flat daily rate, the total charge can **drop sharply** the instant dwell crosses the threshold.

Reproduction with **DEFAULT_ACCESSORIAL_POLICY** (freeTime 120 min, threshold 1440 min = 24 h, detention \$50 / \$150 / \$175 per hour by class, layover \$150/day):

Dwell	Branch	STANDARD	SPECIALIZED	HAZMAT
1440 min (24 h 00 m)	Detention (22 h billable)	\$1,100	\$3,300	\$3,850
1441 min (24 h 01 m)	Layover (ceil = 2 days)	\$300	\$300	\$300

One extra minute of delay drops the bill by **\$800 to \$3,550**. The "cheaper to dwell longer" zone persists for days (for STANDARD, layover only re-exceeds the forfeited detention after roughly 8 days).

Why it matters:

- **Perverse incentive / gaming.** Whichever party is disadvantaged by detention can push dwell just past 24 h to collapse the charge. Billing that *decreases* with more delay is not a defensible model.
- **Revenue and fairness.** The platform under-bills the 1–7 day dwell band and forfeits legitimately-accrued detention.
- **Secondary:** `ceil(dwell / 1440)` bills a full second layover day at 24 h 01 m. That "started periods" convention may be intended, but combined with the discontinuity it amplifies the surprise.

Current exposure: the accessorial billing pipeline is built (services + tables + tests) but not yet broadly switched on, so today's financial blast radius is limited. This should be fixed **before** accessorial billing is enabled for real settlements.

COA:

1. Make the charge **monotonic**: bill `charge = max(detention_equivalent_at_dwell, layover)` — or bill detention up to the threshold **plus** layover for full days beyond it. Product decides the model; both are monotonic.
2. Add a **policy guardrail** asserting `layover(threshold+1) ≥ detention(threshold)` so a misconfigured policy cannot reintroduce the cliff.
3. Add a **property test**: charge is non-decreasing in dwell across the threshold for every rate class and the full policy bound ranges.

F2 — MEDIUM — Operational-capacity math uses floating-point pounds

Where: `backend/src/services/capacityService.ts:13` (`calcMaxOperationalWeight`) and the remaining-weight math that consumes it (lines 76, 102, 111).

What: `calcMaxOperationalWeight(maxCapacityLbs, bufferPct) = maxCapacityLbs * (1 - bufferPct / 100)` returns a **non-integer** whenever `maxCapacityLbs * bufferPct` is not divisible by 100 (e.g. `45001 * 0.9 = 40500.9` , `10001 * 0.85 = 8500.85`). That fractional value then flows into:

- `remainingWeightLbs = maxOpWeight - projectedWeight` (fractional pounds surfaced to the UI),
- the DANGER/WARNING zone gate `projectedTotal > maxOperational` (a floating-point comparison at the boundary),
- the user-facing "Action Denied ... remaining operational capacity of {n} lbs" message, which can render fractional pounds.

Why it matters: it violates the platform's **integer-whole-pounds invariant**. The sibling service `haulerCapacityService.ts` is integer-clean (`Number.isInteger` guard, `Math.max(0, rated - onboard)`), so the two capacity engines now disagree on the invariant, and the float path is the one behind the booking gate.

COA: `Math.floor` the operational weight (flooring is the conservative choice for a safety buffer — you never permit above the buffer), keep every downstream capacity value integer, and add a `Number.isInteger` assertion plus a test with an odd `maxCapacityLbs` .

F3 — LOW — AML gate holds `pending` even when `AML_REQUIRED` is off

Where: `backend/src/services/verification.ts:98` .

What: `amlOk = amlStatus === 'pass' || (!amlRequired() && amlStatus === undefined)` . With AML disabled, an `undefined` status passes (the intended inert M1 behavior) but a `pending` status does **not** — it holds verification. Screening only runs when `amlRequired()` , so `pending` should not arise while AML is off; a legacy/edge `pending` would, however, be held where pre-M1 it would have verified.

Why it matters: negligible today (no path sets `pending` while AML is off), but it is an asymmetry worth removing before AML activation so the flip has no surprise holds.

COA: treat `pending` the same as `undefined` when `!amlRequired()` (both pass while inert), or confirm no code path can set `pending` with AML off and leave a test asserting it.

F4 — LOW — Stale comments contradict the SEC-C1 fix (regression trap)

Where: backend/src/types/platformRole.ts:54-55 (JSDoc) and backend/src/middleware/auth.ts:76 .

What: Both comments still say a missing platformRole resolves to STAFF_ADMIN "for back-compat." The code does the opposite and correct thing (if (stored == null) return null), which is the SEC-C1 CRITICAL fix. The inline SEC-C1 note below the JSDoc is accurate, but the surrounding prose contradicts it.

Why it matters: a future maintainer "cleaning up" the code to match the stale comment would silently reopen the self-signup-ADMIN privilege escalation. Security-relevant comments must not describe the pre-fix behavior.

COA: update the JSDoc and the auth.ts:76 comment to state that a missing/unknown platformRole resolves to null and is denied by the auth gate.

5. Environment parity

Parity matrix

Dimension	Production	Staging	Dev
Backend code	#0c8bf4b (current)	Same build exists; env PAUSED	n/a (no EB)
Frontend code	Behind main (pre- #110)	#110 (deployed this cycle)	n/a
DynamoDB tables	All present; +2 Canopy tables declared-not-applied	Missing CapacityStateEvents	Paper stack (0 live)
Runtime env config	Out-of-band (ignore_changes , by design)	EB env-var drift (in-place plan change)	Explicit overrides
Live health	Green	API down (paused); FE 200	n/a
TF drift (tofu plan)	2 to add (Canopy), 0 change, 0 destroy	1 to add (CapacityStateEvents), 1 change (EB)	n/a

P1 — MEDIUM — Staging is missing the CapacityStateEvents table

tofu plan on envs/staging wants to create module.dynamodb.module.table["CapacityStateEvents"] . The table exists in prod but was never applied to staging (the capacity feature landed after staging was paused). If staging is resumed and the capacity path is exercised, its reads/writes resolve to a LoadLead-Staging-CapacityStateEvents name that does not exist and will fail. COA: targeted apply tofu apply -target='module.dynamodb.module.table["CapacityStateEvents"]' in envs/staging (DynamoDB is on-demand, ~\$0). Do not run a full staging apply (it would also push the EB env-var drift — see P4).

P2 — MEDIUM — Production frontend is behind main

Prod serves an older bundle; main and staging carry the landing mobile-header fix (#110 , 3cd3eb9). This is expected and in-flight (the fix was deployed to staging this cycle, prod promotion pending sign-off). COA: run deploy-frontend.sh (with DEPLOY_MSG set upfront) to close the gap.

P3 — LOW (known/deliberate) — Prod declares 2 unapplied Canopy tables

Prod tofu plan shows ddb_carrier_insurance_connections and ddb_coi_crossreference_results as "to add." Canopy is code-live-but-gated-off in prod (sandbox-first), so these are intentionally not applied. COA: apply them together with the Canopy prod secrets (INF-4) only when Canopy prod is enabled — not before.

P4 — INFO — Staging backend paused; staging EB env drift

api-staging returns no response (HTTP 000 = paused to ~\$0), while the staging FE bucket serves 200. Live staging API smoke is therefore not possible without a resume. Separately, the staging plan shows an in-place EB change — the known out-of-band env-var

drift — which is why staging applies must be `-target` ed at DynamoDB only. **COA:** none required; documented so staging results are interpreted correctly.

6. Verified sound (regression checks on prior rounds)

The following prior-round fixes were re-checked this round and remain correct:

- **SEC-C1** self-signup ADMIN — `resolvePlatformRole(null) → null`, auth gate denies (F4 is only the stale comment).
 - **SEC-C2 / IDOR cluster** — all probed protected paths 401 live.
 - **BL-C1** pagination — (not re-exercised this round; covered by suite).
 - **V2-H1** money-ledger idempotency — conditional puts present on reconciliation + funding advance.
 - **SEC-6** payout intercepts surfaced in payee routing.
 - **BL-1** COI validity — expiry sweeper + expired flags in place.
 - **M1 (v1)** negotiation accept→assign self-heal — `ensureAssignedAndReleased` + `reconcileAcceptedAssignments` + `snapshotPolicyWithRetry` + `policySnapshotPending` present.
 - **First-accept-wins** — atomic conditional lock.
 - **Maps proxy (SEC-H6)** — 401 unauth + rate-limited.
 - **Webhook signature fail-closed** — Canopy + Didit 401 on unsigned.
 - **JSON 404 (v4)** — confirmed live.
 - **render-check rate limit (v4 H4)** — live at the real path.
-

7. Prioritized action plan (COAs)

Now (this or next cycle):

1. **F1** — fix accessorial non-monotonicity (make charge monotonic + policy guardrail + property test). Land before accessorial billing is switched on broadly.
2. **P2** — promote the frontend to prod (`deploy-frontend.sh`) to close the FE deploy gap.
3. **F4** — correct the SEC-C1 comments (5-minute change; removes a security regression trap).

Soon:

1. **F2** — floor operational-capacity to integer pounds + assertion + test.
2. **P1** — targeted staging apply of `CapacityStateEvents` (so a staging resume exercises capacity correctly).

Before feature activation:

1. **F3** — align `pending` handling ahead of `AML_REQUIRED` activation.
2. **P3** — apply Canopy tables **with** Canopy prod secrets (INF-4) only at Canopy prod enablement.

Gated / unchanged from prior rounds: M1 AML activation (Didit standalone product + compliance sign-off), N3 accessorial policy-accept gating (needs an offer-eligibility primitive).

8. Standing recommendations

- **Money/pounds invariant as a lint or test.** F2 shows float can creep back in. Add a focused test (or a small AST lint) asserting capacity/money helpers return integers.

- **Monotonicity as a first-class property.** Any tiered billing curve (detention/layover, and future accessories) should carry a property test that the total is monotonic in the driving quantity.
 - **Security-comment discipline.** Comments on security-critical resolvers must describe post-fix behavior; a stale one is a latent regression (F4).
 - **Deploy-parity visibility.** Consider a tiny `/api/version` (commit SHA) so backend deploy-vs- `main` parity is a one-request check instead of an EB-label reconciliation.
 - **Keep the monthly parity trio** (env-parity checker + prod `tofu plan` + staging `tofu plan`); it is what surfaced P1 this round.
-

9. Appendix — route surface

- **Backend:** 280 route handlers across 28 route files (124 GET, 129 POST, 12 PUT, 6 PATCH, 9 DELETE), mounted under 20 `/api/*` prefixes.
- **Webhooks (raw-body, pre- `express.json`):** Tally `POST /api/admin/beta/webhook` ; Canopy `POST /api/webhooks/canopy` ; Didit `POST /api/webhooks/didit` .
- **Frontend:** central `request()` client (`src/lib/api.ts`); 8 components issue direct `request / fetch` calls outside the client (minor maintainability note, not a defect).

End of report.